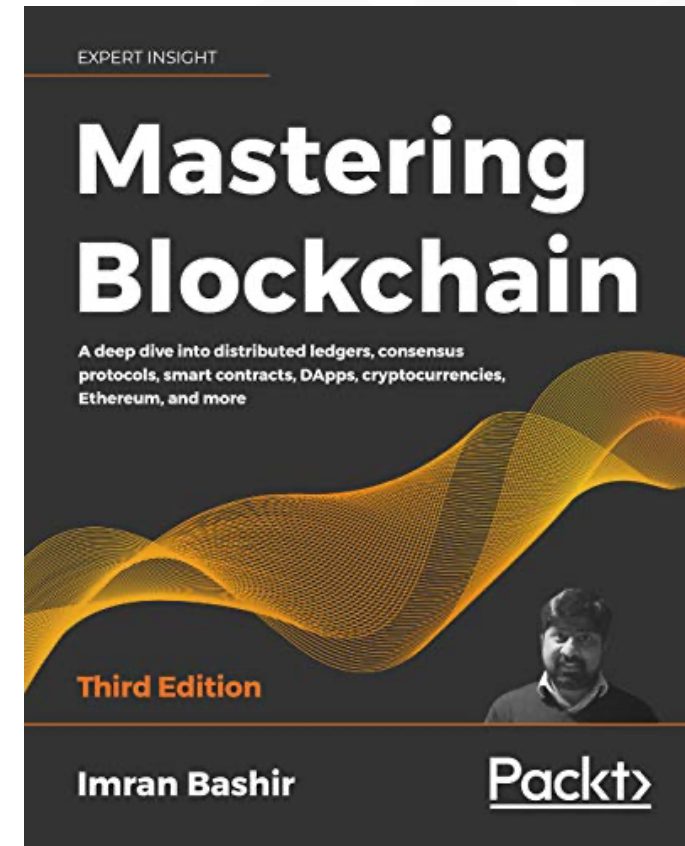


Mastering Blockchain

Third Edition

Chapter 3, Symmetric Cryptography

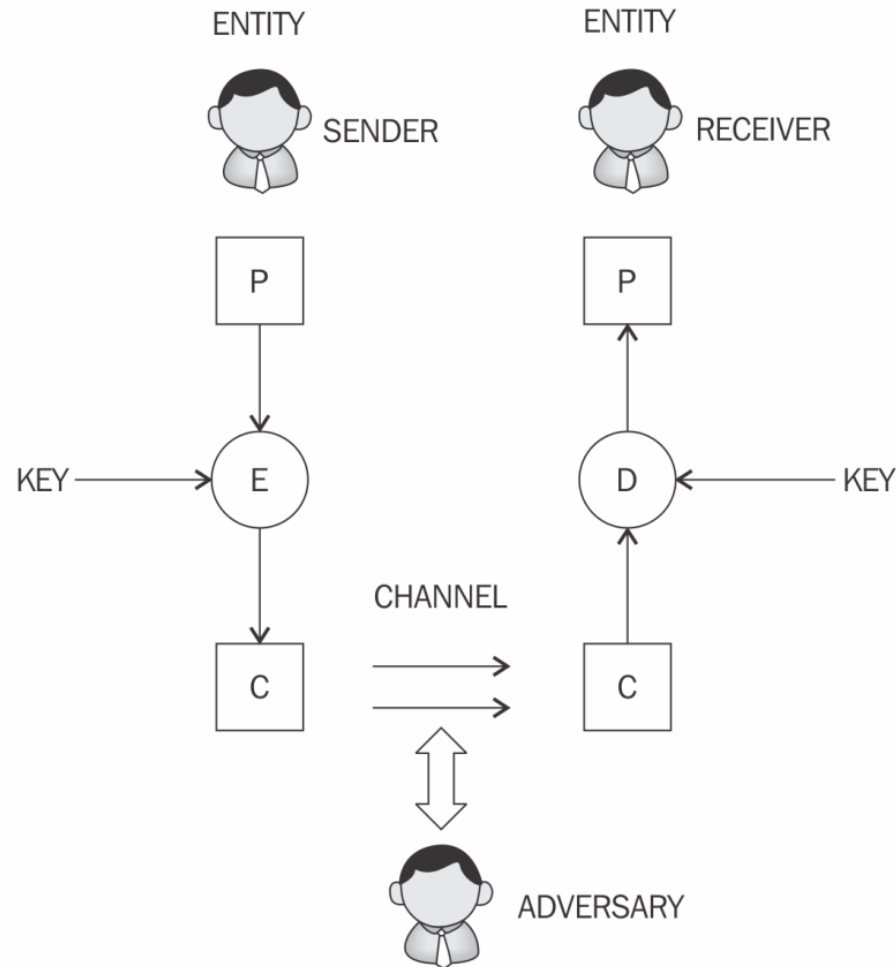


Cryptography

2

A generic cryptographic model is shown here.

P, **E**, **C**, and **D** represent plaintext, encryption, ciphertext, and decryption, respectively.



Cryptography services

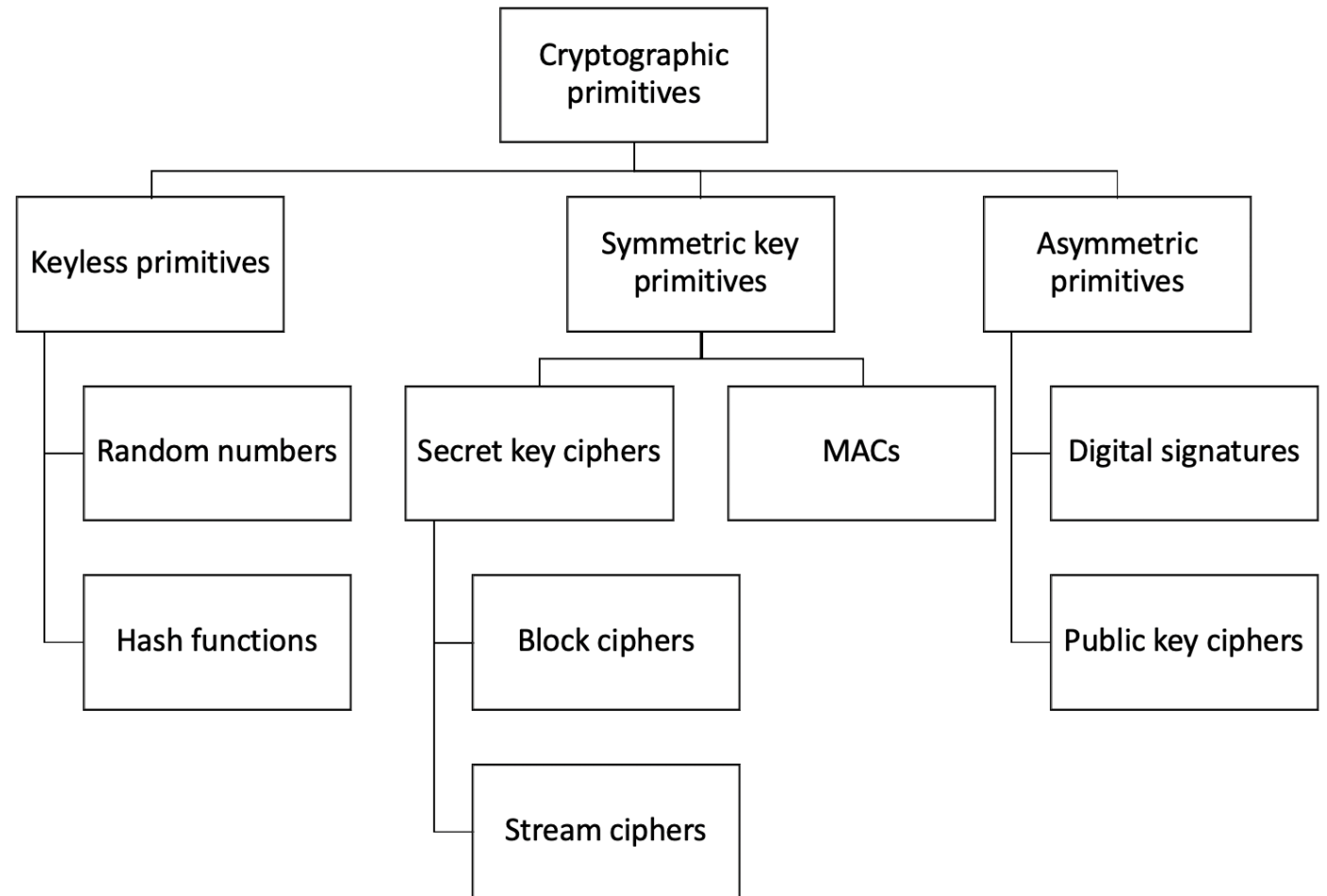
- Confidentiality
- Integrity
- Authentication
- Non-repudiation
- Accountability



Cryptographic primitives

As shown in the preceding cryptographic primitive taxonomy diagram, cryptography is mainly divided into three categories:

- Keyless primitives
- Symmetric key primitives/cryptography
- Asymmetric key primitives/cryptography



Keyless primitives

- Random numbers
- RNGs
- PRNGs
- Hash functions



RNG AND PRNG

- A Random Number Generator (RNG) algorithm is a method used to produce a sequence of numbers that appear to be random. There are two main types:
- **Pseudorandom Number Generators (PRNGs):** These are deterministic algorithms that produce a sequence of numbers that appear random but are generated from an initial "seed" value. If the same seed is used, the same sequence of numbers will be generated. A common example is the Linear Congruential Generator (LCG), which uses a formula like:
 - $X_{i+1} = (aX_i + c) \bmod m$
 - where: X is the sequence of pseudorandom numbers, m is the modulus parameter, a is the multiplier, c is the increment, and X_0 is the initial seed value.

TRNGs



- True Random Number Generators (TRNGs): These rely on physical phenomena that are inherently unpredictable and chaotic to generate truly random numbers. Examples include atmospheric noise, radioactive decay, or fluctuations in voltage. These generators are typically hardware-based.

How PRNG algorithms work:



- PRNGs start with a seed value. This seed can be derived from various sources like the current time, mouse movements, or other unpredictable system events. The algorithm then applies mathematical operations to this seed to produce the next "random" number in the sequence. The key is that while the numbers appear random, they are entirely determined by the initial seed and the algorithm itself. To ensure a different sequence each time, the seed value is often updated continuously.

Applications:

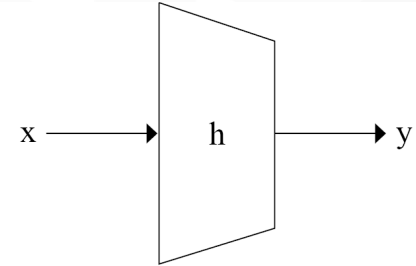
- RNG algorithms are crucial in various fields, including:
- Gaming: Ensuring fairness and unpredictability in games like dice rolls, card shuffles, and lottery draws.
- Cryptography: Generating keys and other random values for secure communication.
- Simulations: Modeling complex systems where random events play a role.
- Scientific research: Statistical sampling and data analysis.

Hash functions

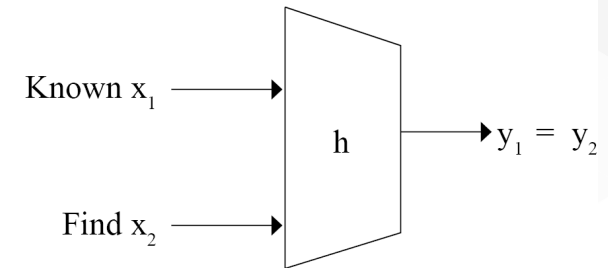
Hash functions compress arbitrary messages into fixed-length digests. They are easy to compute.

The security properties of hash functions are:

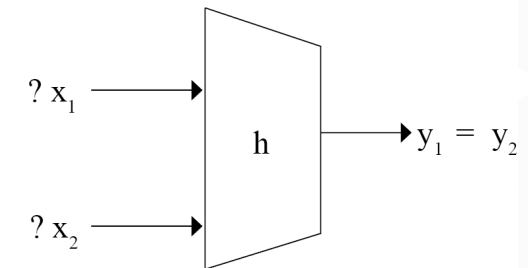
1. Pre-image resistance
2. Second pre-image resistance
3. Collision resistance



1 - PRE-IMAGE RESISTANCE



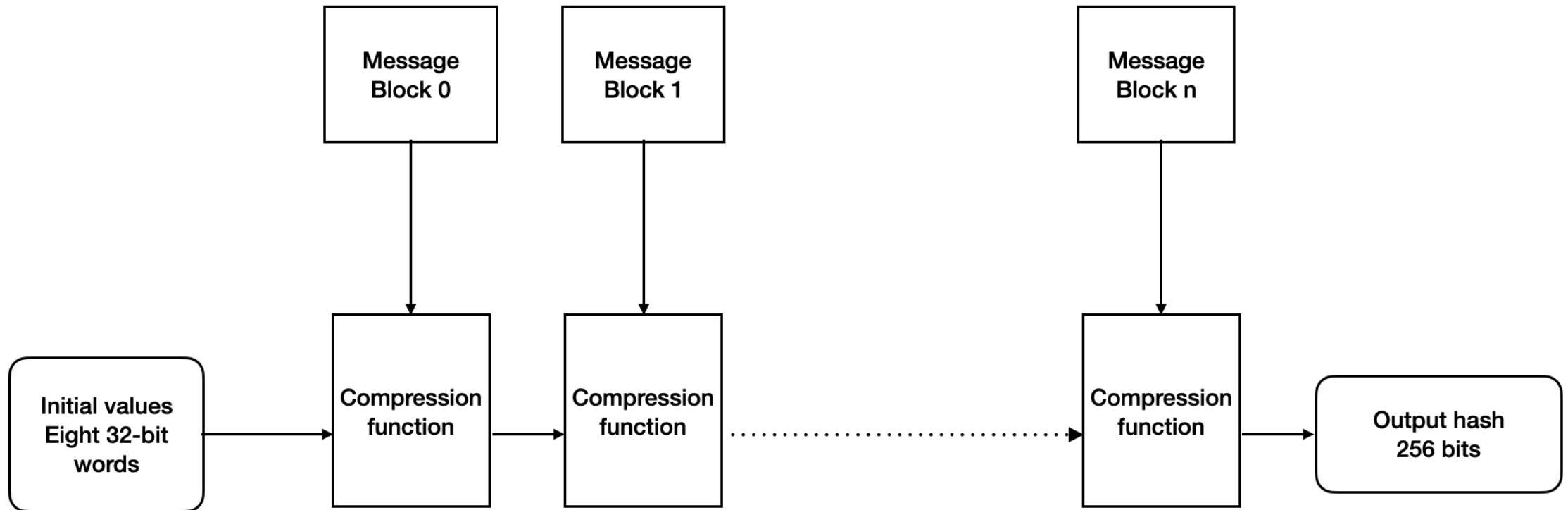
2 - SECOND PRE-IMAGE RESISTANCE



3 - STRONG COLLISION RESISTANCE

SHA256

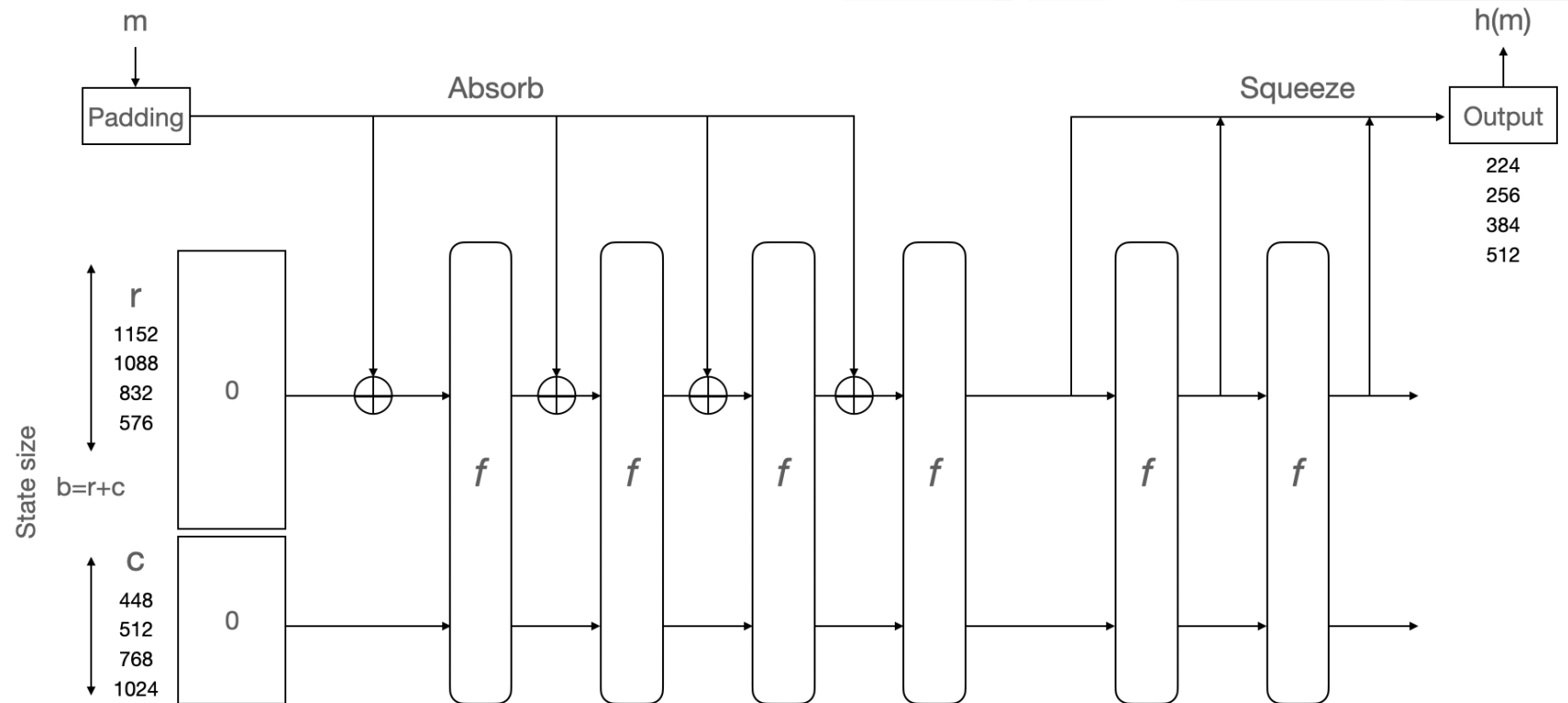
As shown in the preceding diagram, SHA-256 takes the input message and divides it into equal blocks of 512 bits. Initial values (or initial hash values) of the initialization vector are composed of eight 32 bit words (256 bits) that are fed into the compression function with the first message. Subsequent blocks are fed into the compression function until all blocks are processed and finally, the output hash is produced.



Design of SHA-3 (Keccak)

11

The diagram shows the sponge and squeeze model, which is the basis of SHA-3 or Keccak.



f : Permutation function operating on b bits
 r : Bit rate of sponge function (block size)
 c : Capacity bits

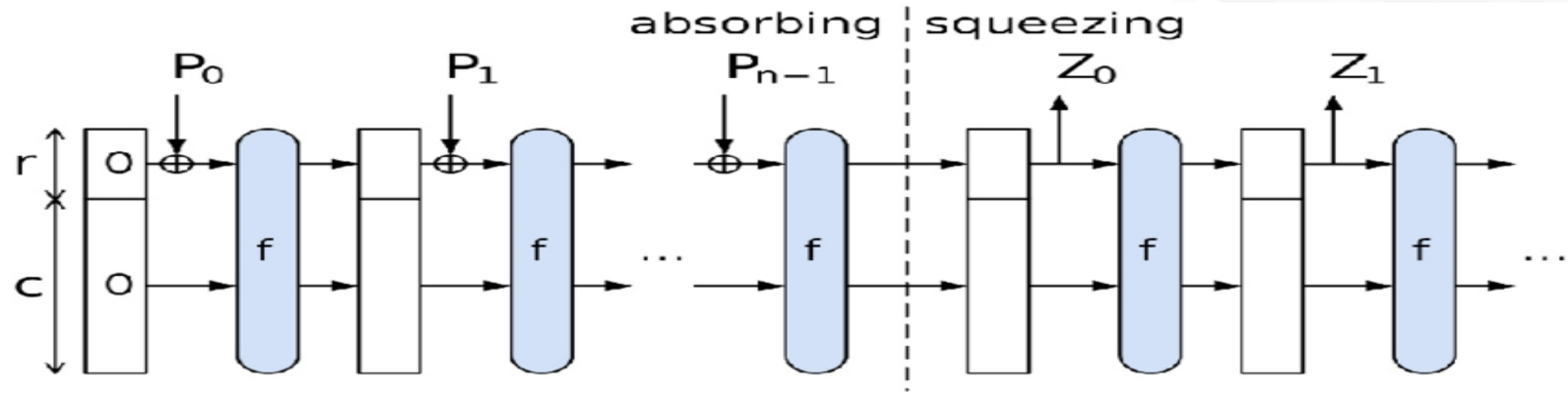
m : Input data
 $h(m)$: Hash of m - Output

f consists of $\theta, \rho, \pi, \chi, \iota$

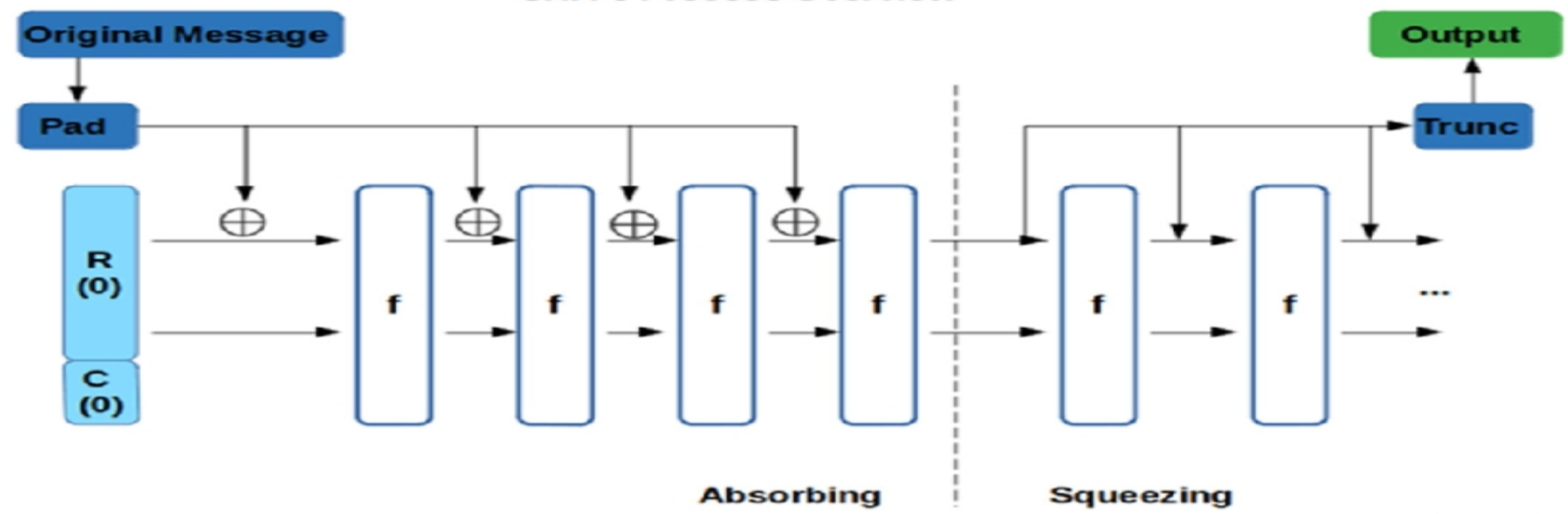
Key aspects of SHA-3:

- **Sponge Construction:**
- This unique design allows SHA-3 to "absorb" input data into a fixed-size internal state and then "squeeze" out a hash value of a desired length. It's like a sponge that can soak up any amount of liquid (data) and then release a specific amount.
- **Keccak Permutation:**
- At the core of the sponge construction is the Keccak permutation function, which applies a series of five sub-rounds (theta, rho, pi, chi, and iota) iteratively to the internal state, ensuring thorough mixing and diffusion of the data.

- **Variable Output Lengths:**
- SHA-3 supports various hash output lengths, including SHA3-224, SHA3-256, SHA3-384, and SHA3-512, which produce hash values of 224, 256, 384, and 512 bits, respectively.
- **Extendable-Output Functions (XOFs):**
- The SHA-3 family also includes SHAKE128 and SHAKE256, which are extendable-output functions. These functions can produce an output of arbitrary length, making them suitable for applications like key derivation and random number generation.
- **Security Properties:**
- SHA-3 is designed to provide strong security properties, including collision resistance (making it computationally infeasible to find two different inputs that produce the same hash output) and preimage resistance (making it computationally infeasible to find the input that produced a given hash output).



SHA-3 Process Overview



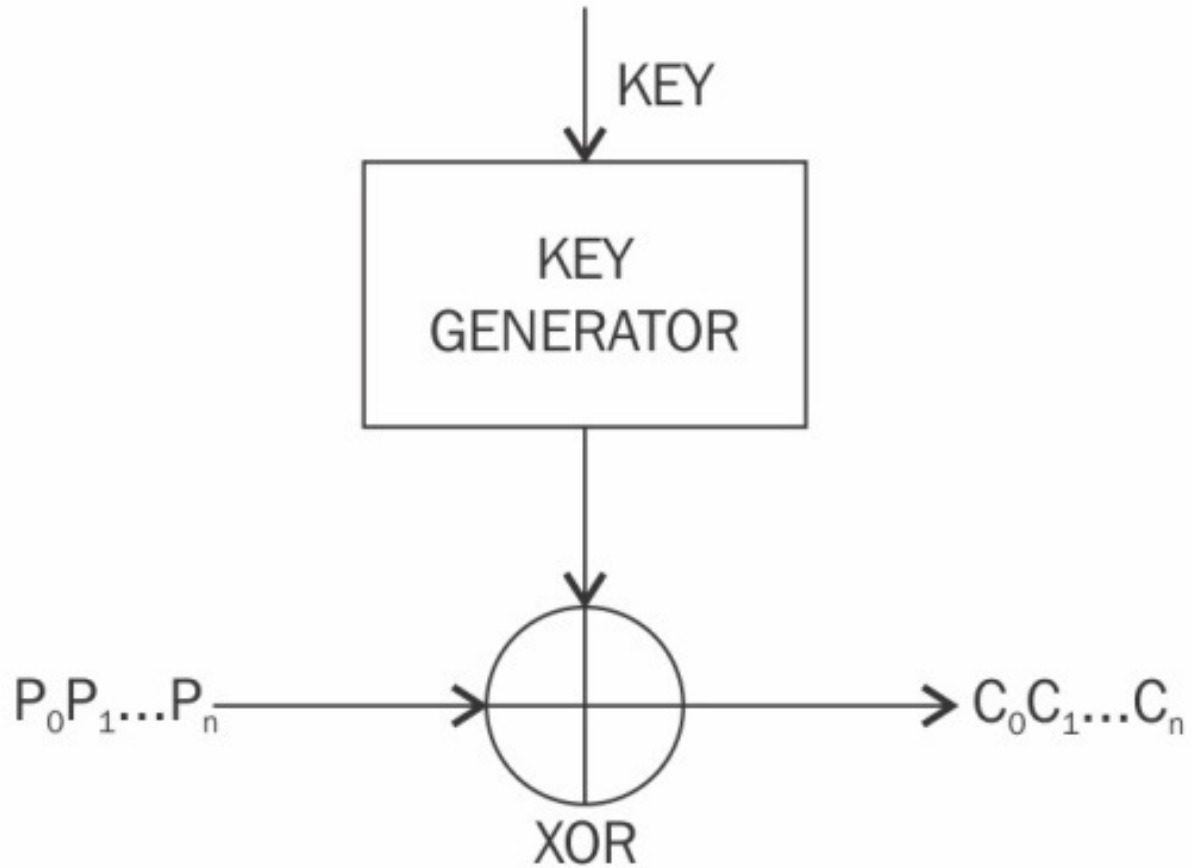
Symmetric cryptography

15

- Uses the same key for encryption and decryption
- Types of keys
 - Public key
 - Private key
 - Ephemeral key
 - Master key

Stream ciphers

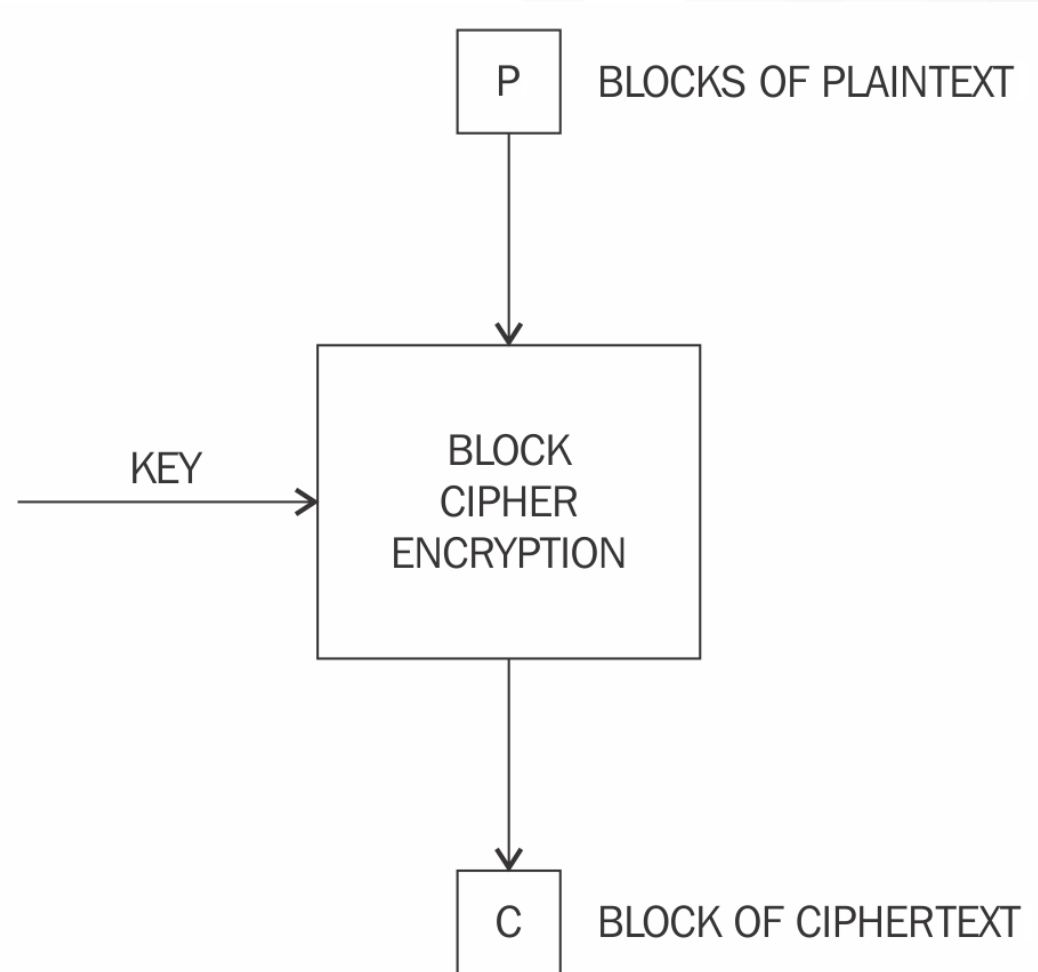
Stream ciphers are encryption algorithms that apply encryption algorithms on a bit-by-bit basis (one bit at a time) to plaintext using a keystream.



Operation of a stream cipher

Block ciphers

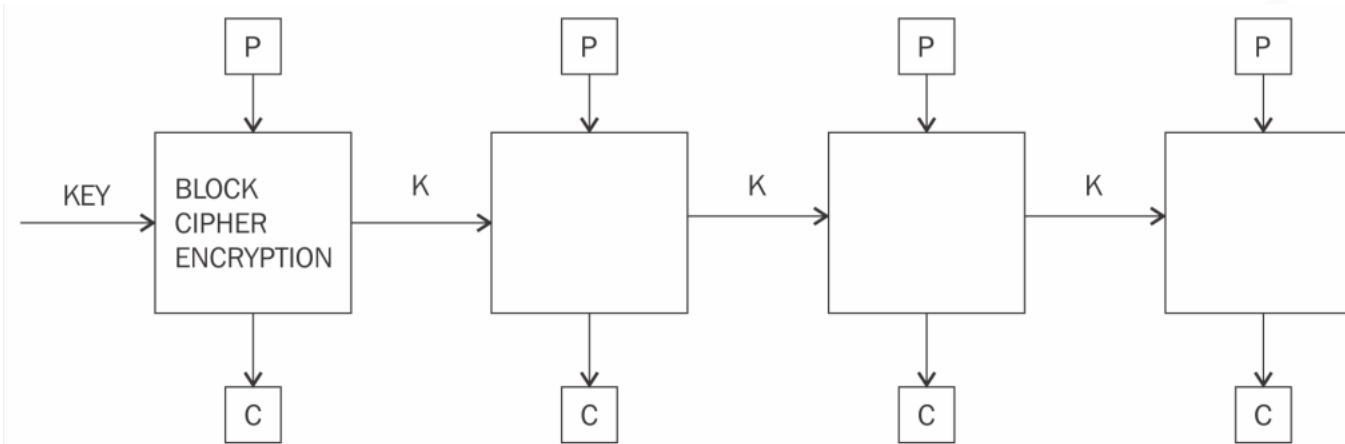
Block ciphers are encryption algorithms that break up the text to be encrypted (plaintext) into blocks of a fixed length and apply the encryption block by block.



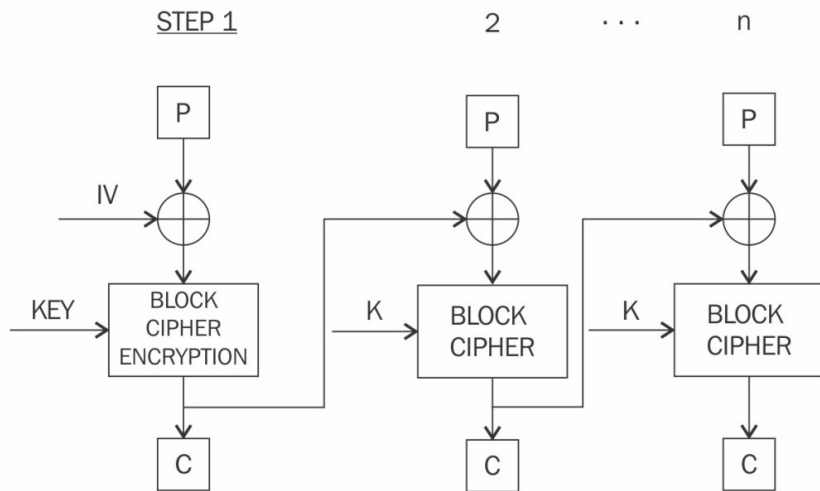
Operation of a block cipher

Block encryption modes

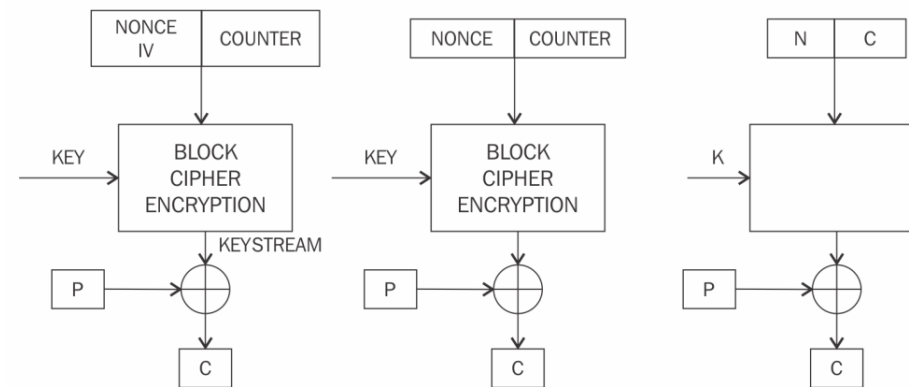
18



Electronic codebook mode

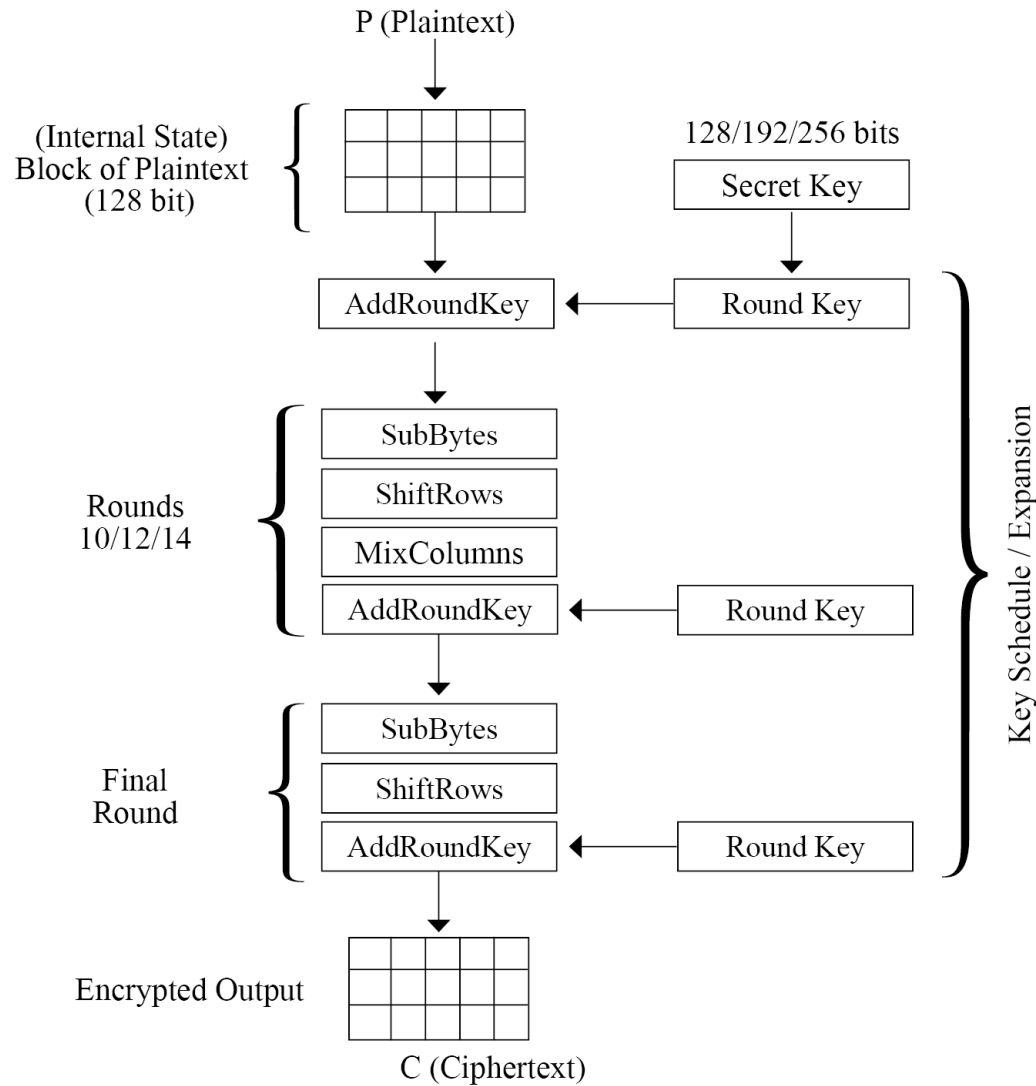


Cipher block chaining mode



Counter mode

Advanced Encryption Standard (AES)



After state initialization four operations are performed in sequence to encrypt the input:

1. **AddRoundKey**: In this step, the state array is XORed with a subkey, which is derived from the master key.
2. **SubBytes**: This is the substitution step where a lookup table (S-box) is used to replace all bytes of the state array.
3. **ShiftRows**: This step is used to shift each row to the left, except for the first one, in the state array in a cyclic and incremental manner.
4. **MixColumns**: Finally, all bytes are mixed in a linear fashion (linear transformation), column-wise.

This is one round of AES. In the final round (either the 10th, 12th, or 14th round, depending on the key size), stage 4 is replaced with **AddRoundKey** to ensure that the first three steps cannot be simply reversed.

Exercise

20

- Search and find a way to encrypt plaintext into an AES 256-bit encrypted output, using the OpenSSL command line tool.

Summary

In this presentation, we...

- Explored symmetric key cryptography.
- Considered some cryptographic primitives.
- Looked at stream and block ciphers.
- Covered the working modes of block ciphers.
- Learned about AES